



本小题结束，请输入End继续...



彩球游戏 magic_ball

の战斗体验报告

班级：计拔/高程05班

学号：2351520

姓名：毛星博

战斗开始时：2024年6月3日晚

战斗胜利时：2024年6月7日晚

崩溃次数：记不清了



微笑中透露着同心同德同
舟楫济人济世济天下

1. 题目：彩球游戏magicball的实现

1.1. 菜单实现模式显示和选择

在运行程序时清屏并显示菜单1-9选项和0退出选项，用户输入对应数字选择模式或输入0选择退出。

1.2. 功能实现9种模式的要求

模式1：生成内部数组，显示其初始状态，如有可消除项则用颜色标记

模式2：生成数组初始状态，寻找可消除项后用不同颜色标记，按回车消除，再按回车非零项下落并用0填充，再按回车在0值处随机生成新的值填充，如此往复直到填充新值后无可消除项

模式3：生成数组初始状态，寻找可消除项后用不同颜色标记，按回车消除，再按回车非零项下落并用0填充，再按回车在0值处随机生成新的值填充，如此往复直到填充新值后无可消除项后，找出可交换项的提示并用不同颜色标记

模式4：画出指定行列数的无分隔线框架并画出彩球的初始状态

模式5：画出指定行列数的有分隔线框架并画出彩球的初始状态

模式6：画出指定行列数的无分隔线框架并画出彩球的初始状态，找出可消除项并用黑色实心球标记

模式7：画出指定行列数的有分隔线框架并画出彩球的初始状态，找出可消除项并用黑色实心球标记，按回车消除可消除的彩球，并使可下落的彩球下落，再按回车则在空白处随机填充新的彩球，如此循环往复直到没有可消除项，显示可交换项的提示并用双圈小球标记

模式8：画出指定行列数的有分隔线框架并画出彩球的初始状态，找出可消除项并用黑色实心球标记，消除可消除的彩球，并使可下落的彩球下落，在空白处随机填充新的彩球，如此循环往复直到没有可消除项，显示可交换项的提示并用双圈小球标记。标记后，移动鼠标可以显示鼠标的行列数位置（如果没有鼠标在彩球的位置，则显示位置非法），鼠标左键点击非可交换项时，显示该球不可选中，如果鼠标左键点击可交换项时选中。鼠标在合法位置点击右键退出程序

模式9：画出指定行列数的有分隔线框架并画出彩球的初始状态，找出可消除项并用黑色实心球标记，消除可消除的彩球，并使可下落的彩球下落，在空白处随机填充新的彩球，如此循环往复直到没有可消除项，显示可交换项的提示并用双圈小球标记。标记后，移动鼠标可以显示鼠标的行列数位置（如果没有鼠标在彩球的位置，则显示位置非法），鼠标左键点击非可交换项时，显示该球不可选中；鼠标在合法位置点击右键退出程序；当鼠标选中可交换项并交换两个彩球后，图形进行消除-下落-填充的动画直到下一次显示提示。在第一次交换开始，统计消除的彩球个数并累计分数。直到再无可交换项时退出程序

2. 整体设计思路

以菜单项9为例。

首先，需要构造一个9*9的int型数组来储存数据，我认为，这个内部数组是整个程序最基础的也是最核心的部分，无论是消除、置零、下落还是寻找可消除项和提示可交换项，本质上都是对这个数组进行操作，而彩球的图形和各类动画只是对这个数组变化的可视化而已。

按照所需的行列数生成初始数组后，画出有分隔线的框架（这里调用画框架的函数，有无分隔线通过参数区分即可），再根据数组填充对应颜色的小球（这里调用画彩球的函数，同样有无分隔线通过参数区分）。

生成初始状态后，对内部数组寻找可消除项（这里调用寻找可消除项的函数，用数组记录可消除项的位置），根据可消除项的位置，将对应位置的小球用黑色实心球标记（此处调用标记函数，此标记函数同时可用于标记数组的可消除项、标记0项等功能，不同功能差异通过参数区分）。

随后，对内部数组可消除项置0（该功能各菜单项使用频率很高，因此整合成函数），同时图形界面标记实心可消除的球进行消除操作（为了消除动画的美观，彩球的爆炸需要有闪烁效果），待所有球消除完后，对内部数组进行“下落”操作，同时可下落的彩球进行下落（为了下落动画的美观，不能一格一格的下落，应该做到一行一行的下落；同时为了下落逻辑正确，考虑采取从下往上一列一列的检索）。这里注意到，为了确保彩球颜色和内部数组的数据能够正确对应，采取同时对内部数组和彩球图形界面操作的方法，即对每一次下落，内部数组该数据下落的同时，对对应的彩球进行下落操作。如果采用整体对内部数组下落后再对图形界面操作，获取小球颜色对应的数据将会非常麻烦。

下落过程结束后，对内部数组0的位置随机填充新值，再根据新值对图形界面填充对应颜色的彩球。如果填充新值后还有可消除项，则再进行消除-下落-填充的过程，直到无可消除项。这个过程可能进行很多次，因此需采用循环逻辑，循环条件为内部数组中存在可消除项。为了便于实现该功能，在设计寻找可消除项的函数时，采用返回布尔值的方式确定是否存在可消除项。

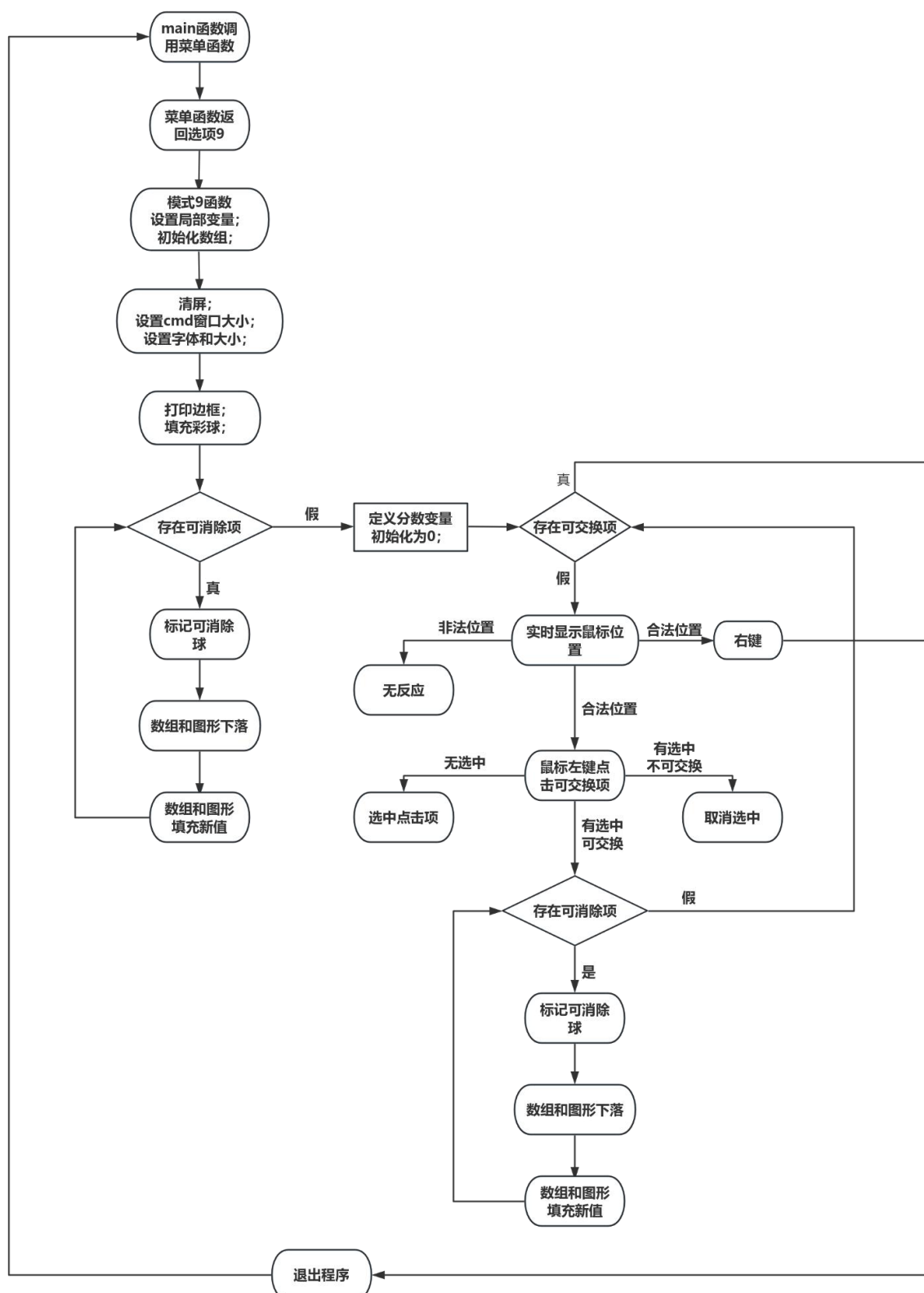
当不存在可消除项时，跳出循环，进入提示环节（寻找可交换项）。这里调用寻找可交换项的函数，将可交换项的位置存入一个提示数组。然后，根据提示数组，将图形界面的对应的可交换的彩球用双圈球标记。

标记完可交换项后，进入鼠标操作环节。首先用函数，允许使用鼠标。然后，用循环逻辑，即时获取鼠标事件。获取到鼠标的位置后，计算并判断鼠标是否位于合法位置，实时输出鼠标的行列位置或位置非法提示。如果接收到鼠标左键点击，如果鼠标位于可交换小球的位置，则选中该球。为了实现“选中”的效果，函数中需要定义两个变量用于存放选中小球的位置。当再次获取到点击鼠标左键事件后，如果点击位置是上次选中的位置，则取消选中。如果点击不可交换项则无反应，如果点击到可交换项，但与选中的球不匹配，则取消选中并提示不可交换。如果点击匹配的可交换项，则内部数组交换数值，同时图形界面彩球交换位置。再次进入判断消除-下落-填充新值的循环直到无可交换项。同时需要定义一个变量统计分数。

每次执行完一次交换-消除-下落-填充的过程后，重新寻找内部数组中可交换项。直到无可交换项或鼠标在合法位置点击右键退出程序。

3. 主要功能的实现

以菜单项9为例。



4. 调试过程碰到的问题

问题1：如何高效、准确地判断出可交换项？

解决方法：一开始，我采取的是PDF中提示的方法，对每一个位置，搜索其周围是否有符合条件的数据。但是我发现这样编写的代码需要考虑较多情况，并且容易遗漏情况。为了简化逻辑，我采取了另外一种判断方式，我将其称之为模拟移动法。对规定行列数的数组，对于每一次可能的横向或者竖向交换，都创建一个数组副本，这个副本是单次交换之后的。然后，只需要判断这个数组是否存在可消除项即可。该方法逻辑简单，不会遗漏情况，且可以使用现成的函数——判断是否存在可消除项。但也存在缺点——判断需要遍历每一种交换情况，需要的计算量比常规方法更大，处理效率降低。

问题2：如何让图形界面的变化动画更加流畅自然，比如小球的爆炸、下落动画

解决方法：首先，为了让动画流畅，需要设置合适的延时。为了显示出动画爆炸的效果，我采取了循环播放两种图形的方式，使得动画有闪烁的效果。同时，为了使动画更流畅，下落时不能够一格一格的改变小球的位置，否则会显得很僵硬。应该对下落的每一行进行消除-重现，即小球可以出现在边框的位置。为了显示出下落的正确效果，我采取自下往上每列遍历的方式，实现了demo的下落效果。

问题3：在菜单项9的调试过程中，在合法位置按右键能够成功退出，但是cmd窗口会出现弹窗，如何解决这个问题

解决方法：这个问题困扰了我很长时间，主要是因为我无法确定是那部分代码造成了这种情况。因此，采取对每部分代码分开调试的方式，首先确定造成问题的代码块。首先我怀疑是获取鼠标事件的代码造成，所以我尝试在非法位置按下右键，结果不会有弹窗，因此排除。然后在群里交流之后，我认为可能是因为等待输入end的函数中的输入函数fgets有问题，但是我将其替换成cin、getline等函数仍然没有解决问题。最后没办法的情况下，我使用对每一个语句加注释在调试的方法，最终发现去掉最后禁用鼠标操作的函数，即可解决弹窗问题。

5. 心得体会

5.1 完成本次作业得到的一些心得体会、经验教训

熟悉题目要求：在开始作业之前，认真阅读和理解每个题目的要求非常重要。尤其是在复杂题目中，每一个细节都可能影响解题思路。比如，有些题目可能会有隐藏条件或者特殊情况，需要在解题前彻底理解题目才能避免这些陷阱。花时间理解题目不仅可以减少错误，还可以帮助我制定更有效的解题策略。

规划解题步骤：对于复杂的问题，提前规划好解题步骤，明确每一步需要完成的任务，可以提高效率并减少出错的概率。例如，可以先写一个伪代码或者流程图来帮助理清思路。通过这种方式，可以在动手编写代码之前就发现可能存在的问题，从而避免在编写过程中反复修改。比如，在写图形界面下落的函数时，我画了一个流程图来明确每一步的需求。这种方式可以大大提高解题效率，明确目标，并且这种方式有时候可以脱离电脑单独进行。

调试与验证：在完成每个小题后，及时进行调试与验证，确保每一步的结果都是正确的。这可以避免后续步骤因为前面的问题而出现偏差。调试时可以使用打印语句或者断点调试工具，逐步检查每个函数的输出是否符合预期。验证过程中，可以设计一些边界测试用例，确保代码在各种情况下都能正常运行。比如这次大作业的调试过程，我使用了vs2022的调试工具，合理设置断点，使用逐语句调试方法，时时关注变量的值，从而迅速发现问题。同时，可以使用分布打印内部数组的方式来解决图形界面的许多问题。同时，我编写了一个暂停函数，等待回车输入才会继续执行。这个方法可以有效提高找出问题的效率，将代码分区测试。

总结与反思：在完成作业后，进行总结与反思，分析在解题过程中遇到的问题以及如何解决这些问题，对于提高解题能力非常有帮助。通过总结，我们可以发现自己的不足之处，并在以后加以改进。反思可以帮助我们积累经验，避免在未来的作业中重复犯相同的错误。通过不断的总结和反思，可以逐步提高自己的编程能力和解题水平。

5.2 通过综合题 1/2 中有关函数的分解与使用，总结你在完成过程中是否考虑了前后小题的关联关系，是否能尽可能做到后面小题有效利用前面小题已完成的代码，如何才能更好地重用代码？

在第二次完成大作业时，我对函数的分解、使用的能力明显比第一次大作业时有所精进。对于某些功能，我能够较为快速的提取出需要整合的函数，比如标记函数、判断是否存在可消除项函数等，并且能够正确设置函数的返回类型以便后续编写。比如是否存在可消除项函数，返回布尔类型，能够配合后面的判断可交换项函数一起使用。

考虑小题的关联关系：在解题过程中，我考虑了前后小题之间的关联关系。例如，如果某个小题的结果可以用于后续小题，我会尽量将其设计为独立的函数，以便后续调用。通过这种方式，不仅可以减少重复劳动，还可以确保后续步骤的正确性。这样，当前一个小题的解答出现问题时，只需修改该小题的函数即可，无需重新调整整个大型函数的代码。

代码重用：为了提高代码的重用性，我将一些通用的功能提取出来，编写成独立的函数。例如，画边框、填充彩球等，这样在后续的小题中只需要调用这些函数，而不需要重复编写相同的代码。这不仅可以提高工作效率，还可以确保代码的一致性和可维护性。重用代码时，需要特别注意函数的输入输出和边界条件，确保函数的通用性和可靠性。

模块化编程：将复杂的问题分解为多个模块，每个模块负责完成特定的功能。通过这种方式，不仅可以提高代码的可读性，还可以方便调试与维护。模块化编程可以使代码结构更加清晰，便于理解和修改。每个模块可以独立开发和测试，从而减少整体调试的难度。当某个模块需要修改时，只需在该模块内部进行调整，而不会影响到其他部分的代码。

函数的命名与注释：在编写函数时，我注意为每个函数取一个有意义的名字，并添加必要的注释。这样不仅可以提高代码的可读性，还可以方便自己和他人在后续阅读、使用这些函数时快速理解其功能。函数命名要简洁明了，能够准确描述函数的功能。注释不仅要说明函数的用途，还要解释函数的参数、返回值和内部逻辑，确保代码在长时间后仍然易于维护。

6. 附件：源程序

```
bool can_disappear(int shuju[9][9], int panduan[54][4]) //判断是否有可以消除的组，返回布尔值真或假
{
    bool flag = false;
    int h = 0;
    for (int i = 0; i < 54; i++) //每次判断都初始化，避免上次干扰
        for (int j = 0; j < 4; j++)
            panduan[i][j] = 0;
    for (int i = 0; i < 9; i++)
        for (int j = 0; j < 9; j++) {
            if (shuju[i][j] != 0) {
                panduan[h][2] = 1;
                for (int k = 1; i + k < 9 && shuju[i + k][j] == shuju[i][j]; k++)
                    panduan[h][2]++;
                if (panduan[h][2] >= 3) {
                    panduan[h][3] = 2;
                    panduan[h][0] = i;
                    panduan[h][1] = j;
                    flag = true;
                    h++;
                }
            }
        }
    if (shuju[i][j] != 0) {
        panduan[h][2] = 1;
        for (int k = 1; j + k < 9 && shuju[i][j + k] == shuju[i][j]; k++)
            panduan[h][2]++;
        if (panduan[h][2] >= 3) {
            panduan[h][3] = 1;
            panduan[h][0] = i;
            panduan[h][1] = j;
            flag = true;
            h++;
        }
    }
}
return flag;
}
```

```
void hint(int shuju[9][9], int tishi[144][4], int hang, int lie, int& t) //找出可交换项并存入 tishi 数组
{
    int shuju_virtual[9][9] = { 0 };
    int panduan_virtual[54][4] = { 0 };
    int exchange;

    t = 0; //每次重置提示数组
    for (int i = 0; i < 144; i++)
        for (int j = 0; j < 4; j++)
            tishi[i][j] = 0;

    for (int i = 0; i < hang; i++) { //模拟横向交换
        for (int j = 0; j < lie - 1; j++) {
            for (int i0 = 0; i0 < hang; i0++)
                for (int j0 = 0; j0 < lie; j0++)
                    shuju_virtual[i0][j0] = shuju[i0][j0];
            exchange = shuju_virtual[i][j];
        }
    }
}
```

```

        shuju_virtual[i][j] = shuju_virtual[i][j + 1];
        shuju_virtual[i][j + 1] = exchange;
        if (can_disappear(shuju_virtual, panduan_virtual)) {
            tishi[t][0] = i;
            tishi[t][1] = j;
            tishi[t][2] = i;
            tishi[t][3] = j + 1;
            t++;
        }
    }
}
for (int i = 0; i < hang - 1; i++) { //模拟纵向交换
    for (int j = 0; j < lie; j++) {
        for (int i0 = 0; i0 < hang; i0++)
            for (int j0 = 0; j0 < lie; j0++)
                shuju_virtual[i0][j0] = shuju[i0][j0];
        exchange = shuju_virtual[i][j];
        shuju_virtual[i][j] = shuju_virtual[i + 1][j];
        shuju_virtual[i + 1][j] = exchange;
        if (can_disappear(shuju_virtual, panduan_virtual)) {
            tishi[t][0] = i;
            tishi[t][1] = j;
            tishi[t][2] = i + 1;
            tishi[t][3] = j;
            t++;
        }
    }
}
}

void xialuo_graph(int shuju[9][9], int panduan[54][4], int hang, int lie, int& score) //数组和图形同步下落，用于菜单
8/9，同时累计分数
{
    for (int h = 0; panduan[h][2] > 2; h++) { //将数组可消除项置零+图形爆炸㊦
        if (panduan[h][3] == 1) {
            for (int j = panduan[h][1]; j < panduan[h][1] + panduan[h][2]; j++) {
                if (shuju[panduan[h][0]][j] == 0)
                    continue;
                cct_setcolor(shuju[panduan[h][0]][j] + 1, 0);
                for (int m = 0; m < 4; m++) {
                    cct_gotoxy(2 + j * 4, 2 + panduan[h][0] * 2);
                    cout << "O";
                    Sleep(25);
                    cct_gotoxy(2 + j * 4, 2 + panduan[h][0] * 2);
                    cout << "㊦";
                    Sleep(25);
                }
                Sleep(25);
                cct_setcolor(15, 15);
                cct_gotoxy(2 + j * 4, 2 + panduan[h][0] * 2);
                cout << " ";
                cct_setcolor(0, 7);
                shuju[panduan[h][0]][j] = 0;
            }
        }
        else if (panduan[h][3] == 2) {
            for (int i = panduan[h][0]; i < panduan[h][0] + panduan[h][2]; i++) {

```

```

        if (shuju[i][panduan[h][1]] == 0) {
            continue;
        }
        cct_setcolor(shuju[i][panduan[h][1]] + 1, 0);
        for (int m = 0; m < 4; m++) {
            cct_gotoxy(2 + panduan[h][1] * 4, 2 + i * 2);
            cout << "○";
            Sleep(25);
            cct_gotoxy(2 + panduan[h][1] * 4, 2 + i * 2);
            cout << "◐";
            Sleep(25);
        }
        Sleep(25);
        cct_setcolor(15, 15);
        cct_gotoxy(2 + panduan[h][1] * 4, 2 + i * 2);
        cout << " ";
        cct_setcolor(0, 7);
        shuju[i][panduan[h][1]] = 0;
    }
}

for (int i = 0; i < hang; i++) //累计单次消除分数
    for (int j = 0; j < lie; j++)
        if (shuju[i][j] == 0)
            score++;
for (int j = 0; j < lie; j++) {
    for (int i = hang - 1; i > 0; i--) {
        if (shuju[i][j] == 0) {
            int k;
            for (k = i; shuju[k][j] == 0; k--)
                if (k == -1)
                    break;
            if (k == -1)
                break;
            for (; k < i; k++) {
                shuju[k + 1][j] = shuju[k][j];
                cct_setcolor(15, 15);
                cct_gotoxy(2 + j * 4, 2 + k * 2);
                cout << " ";
                cct_setcolor(shuju[k][j] + 1, 0);
                cct_gotoxy(2 + j * 4, 2 + k * 2 + 1);
                cout << "○";
                Sleep(40);
                cct_setcolor(15, 0);
                cct_gotoxy(2 + j * 4, 2 + k * 2 + 1);
                cout << "=";
                cct_setcolor(shuju[k][j] + 1, 0);
                cct_gotoxy(2 + j * 4, 2 + k * 2 + 2);
                cout << "○";
                Sleep(40);
                shuju[k][j] = 0;
            }
        }
    }
}
cct_setcolor(0, 7);
}

```

```

void mouse_full(int shuju[9][9], int tishi[144][4], int hang, int lie, int t, bool& tuichu) //菜单项 9 处理鼠标事件
{
    int MX, MY, MAction, keycode1, keycode2; //用于处理鼠标
    int event;
    int hang_old = -1, lie_old = -1; //记录选中可选项后的位置
    int m; //处理交换

    while (1) {
        event = cct_read_keyboard_and_mouse(MX, MY, MAction, keycode1, keycode2);
        bool flag = false;
        if (event == CCT_MOUSE_EVENT) {
            if (((MX <= 3 + 4 * (lie - 1)) && (MX % 4 == 2 || MX % 4 == 3)) && ((MY > 0) && (MY <= 2 + (hang
                - 1) * 2) && (MY % 2 == 0)))

                flag = true;
            cct_gotoxy(0, 2 + hang * 2);
            // 打印鼠标的实时位置
            if (flag)
                cout << "[当前光标]" << char(MY / 2 + 64) << " 行 " << (MX) / 4 + 1 << " 列";
            else
                cout << "[当前光标]" << "位置非法";
            // 检查鼠标按键事件
            bool flag2 = false;
            if (MAction == 0x0002) { //点击左键
                for (int t0 = 0; t0 < t; t0++) {
                    if ((MX == tishi[t0][1] * 4 + 2 || MX == tishi[t0][1] * 4 + 3) && (MY == tishi[t0][0] * 2 + 2)) {
                        flag2 = true;
                        break;
                    }
                }
                else if ((MX == tishi[t0][3] * 4 + 2 || MX == tishi[t0][3] * 4 + 3) && (MY == tishi[t0][2] * 2 + 2)) {
                    flag2 = true;
                    break;
                }
            }
            cct_gotoxy(0, 2 + hang * 2);
            if (flag2) {
                cout << "当前选择 " << char(MY / 2 + 64) << " 行 " << (MX) / 4 + 1 << " 列";
                if (hang_old == -1) { //未选择可交换项时
                    hang_old = MY / 2 - 1; //从 0 开始
                    lie_old = MX / 4; //从 0 开始
                    cct_setcolor(shuju[hang_old][lie_old] + 1, 15);
                    cct_gotoxy(2 + (lie_old) * 4, 2 + hang_old * 2);
                    cout << "◎";
                    cct_setcolor(0, 7);
                }
                else { //已选择可交换项
                    if ((lie_old == MX / 4) && (hang_old == MY / 2 - 1)) { //再选同一个
                        cct_setcolor(shuju[hang_old][lie_old] + 1, 0);
                        cct_gotoxy(2 + (lie_old) * 4, 2 + hang_old * 2);
                        cout << "◎";
                        hang_old = -1;
                        lie_old = -1;
                        cct_setcolor(0, 7);
                    }
                }
            }
            else {

```

```

for (int t0 = 0; t0 < t; t0++) {
    //选到匹配的可交换项
    if (((tishi[t0][0] == hang_old) && (tishi[t0][1] == lie_old) && (tishi[t0][2] ==
        MY / 2 - 1) && (tishi[t0][3] == MX / 4))
        || ((tishi[t0][2] == hang_old) && (tishi[t0][3] == lie_old) && (tishi[t0][0]
            == MY / 2 - 1) && (tishi[t0][1] == MX / 4))) {
        cct_setcolor(shuju[hang_old][lie_old] + 1, 0);
        cct_gotoxy(2 + (MX / 4) * 4, 2 + (MY / 2 - 1) * 2);
        cout << "◎";
        cct_setcolor(shuju[MY / 2 - 1][MX / 4] + 1, 0);
        cct_gotoxy(2 + (lie_old) * 4, 2 + (hang_old) * 2);
        cout << "◎";
        m = shuju[hang_old][lie_old];
        shuju[hang_old][lie_old] = shuju[MY / 2 - 1][MX / 4];
        shuju[MY / 2 - 1][MX / 4] = m;
        cct_setcolor(0, 7);
        return;
    }
}
for (int t0 = 0; t0 < t; t0++) {
    //选到另一组的可交换项
    if (((tishi[t0][2] == MY / 2 - 1) && (tishi[t0][3] == MX / 4)) || ((tishi[t0][0] ==
        MY / 2 - 1) && (tishi[t0][1] == MX / 4))) {
        cct_setcolor(shuju[hang_old][lie_old] + 1, 0);
        cct_gotoxy(2 + (lie_old) * 4, 2 + hang_old * 2);
        cout << "◎";
        cct_setcolor(0, 7);
        cct_gotoxy(0, 2 + hang * 2);
        cout << "不能交换 " << char(MY / 2 + 64) << "行" << (MX) / 4 + 1 <<
            "列" << " <=> " << char(hang_old + 65) << "行" << lie_old + 1 << "列";
        Sleep(500);
        cct_gotoxy(0, 2 + hang * 2);
        cout << "
";
        hang_old = -1;
        lie_old = -1;
        break;
    }
}
}
}
}
}
else
    cout << "不能选择 " << char(MY / 2 + 64) << " 行 " << (MX) / 4 + 1 << " 列";
}
else if (MAAction == 0x0008 && flag) {
    tuichu = true;
    break;
}
}
}
cout.flush();
}
}
}

```